

Informe de Implementación

Despliegue Continuo Robusto con un Pipeline de CI/CD

usando GitHub Actions

Juan Sebastian Otero Vega - 2220053

Farid Camilo Rojas Vargas - 2220051

Juan David Pallares Pallares - 2220079

Mayo 2026

Link del Código:

https://github.com/JuanSe2731/Proyecto-Spring-Boot-E-converse_vers2.git

Índice

1. Introducción	3
2. Descripción general del sistema	3
2.1. Stack tecnológico	3
3. Estructura del repositorio	3
4. Conceptos fundamentales de CI/CD	4
4.1. Integración Continua (CI)	4
4.2. Entrega Continua (CD)	5
4.3. Conceptos de GitHub Actions	5
5. Estrategia de Branching	5
6. Explicación del Pipeline CI/CD	6
6.1. Job 1: Build & Test	6
6.2. Job 2: Docker Build & Push	7
6.3. Job 3: Deploy	9
6.4. Job 4: Notificaciones	11
7. Pruebas unitarias implementadas	11
7.1. Pruebas del ProductoService	12
7.2. Pruebas del CategoriaService	12
7.3. Pruebas del UsuarioService	12
7.4. Pruebas del JwtTokenUtil	12
7.5. Pruebas del modelo Producto	13
8. Gestión de secretos	13
8.1. Secretos configurados	14
9. Registro de contenedores (Docker Hub)	14
9.1. Estrategia de etiquetado	15
9.2. Dockerfiles (Multi-stage builds)	15
10. Resultados obtenidos	17
10.1. Ejecución del pipeline completo	17
10.2. Detalle de cada job	18
10.3. Notificación recibida	20
11. Análisis y conclusiones	20
11.1. Beneficios del CI/CD implementado	20
11.2. Lecciones aprendidas	21

1. Introducción

En este trabajo se implementa un pipeline de Integración Continua y Entrega Continua (CI/CD) automatizado para la aplicación **E-Converse**, una plataforma de comercio electrónico desarrollada con Spring Boot (backend), React con Vite (frontend) y MongoDB (base de datos).

El objetivo principal es automatizar el proceso completo de entrega de software: desde la compilación y ejecución de pruebas unitarias, pasando por la construcción y publicación de imágenes Docker en Docker Hub, hasta el despliegue en un servidor remoto mediante SSH. Adicionalmente, se implementa un sistema de notificaciones por correo electrónico para informar sobre el estado de cada etapa del pipeline.

La herramienta seleccionada para la implementación del pipeline es **GitHub Actions**, la cual permite definir flujos de trabajo (workflows) directamente en el repositorio de código fuente mediante archivos YAML.

2. Descripción general del sistema

2.1. Stack tecnológico

- **Backend:** Spring Boot 3.5.6 (Java 17), Spring Security con JWT, Spring Data MongoDB
- **Frontend:** React 19 con Vite 7, TailwindCSS 3.4, Zustand para estado global
- **Base de datos:** MongoDB Atlas (NoSQL)
- **Contenedores:** Docker con multi-stage builds
- **CI/CD:** GitHub Actions
- **Registro de imágenes:** Docker Hub

3. Estructura del repositorio

El proyecto se organiza en dos directorios principales: **backend/** para la API REST con Spring Boot y **frontend-react/** para la SPA con React. El pipeline de CI/CD se define en `.github/workflows/ci-cd.yml`.

```

Proyecto-Spring-Boot-E-converse_vers2/
|-- .github/
|   |-- workflows/
|       |-- ci-cd.yml                    # Pipeline CI/CD
|-- backend/
|   |-- Dockerfile                      # Multi-stage build
|   |-- pom.xml                        # Dependencias Maven
|   |-- src/main/java/                 #Codigo fuente
|   |-- src/test/java/                 # Pruebas unitarias
|-- frontend-react/
|   |-- Dockerfile                      # Multi-stage build
|   |-- nginx.conf                     # Config Nginx
|   |-- package.json                   # Dependencias Node
|   |-- src/                           # Codigo fuente React
|-- docker-compose.yml                 # Compose local
|-- informe/                           # Este informe

```

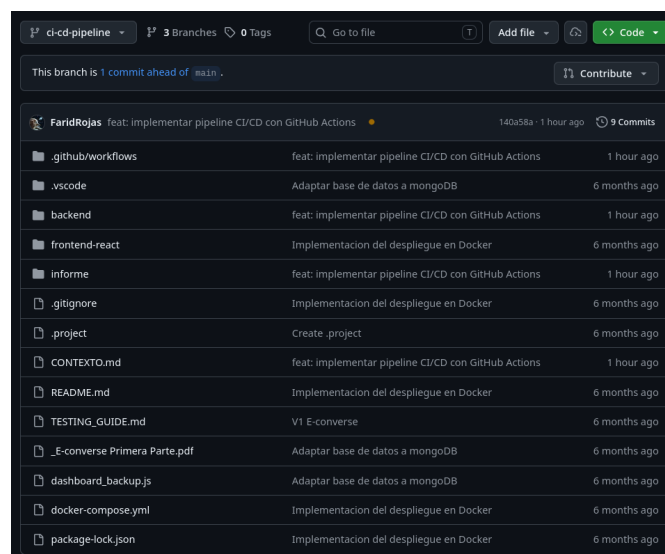


Figura 1: Estructura del repositorio en GitHub

4. Conceptos fundamentales de CI/CD

4.1. Integración Continua (CI)

La Integración Continua es la práctica de fusionar frecuentemente los cambios de código de todos los desarrolladores en una rama compartida. Cada fusión activa un proceso automatizado que compila el código y ejecuta pruebas para detectar errores tempranamente.

4.2. Entrega Continua (CD)

La Entrega Continua extiende la CI automatizando el despliegue del software en entornos de prueba o producción. El objetivo es que cada cambio que pasa las pruebas esté listo para ser desplegado en cualquier momento.

4.3. Conceptos de GitHub Actions

- **Workflow:** Proceso automatizado definido en un archivo YAML dentro de `.github/workflows/`. Se activa por eventos como `push` o `pull_request`.
- **Job:** Conjunto de pasos (steps) que se ejecutan en un mismo runner. Los jobs pueden ejecutarse en paralelo o secuencialmente usando dependencias.
- **Step:** Acción individual dentro de un job. Puede ser un comando shell o una acción predefinida.
- **Runner:** Máquina virtual donde se ejecutan los jobs. GitHub proporciona runners con Ubuntu, Windows y macOS.
- **Artifact:** Archivo generado durante la ejecución del workflow que puede ser descargado o utilizado por otros jobs.
- **Secret:** Variable cifrada almacenada en la configuración del repositorio para proteger credenciales sensibles.

5. Estrategia de Branching

Para este proyecto se utilizó una estrategia basada en **GitHub Flow**, creando una rama específica `ci-cd-pipeline` desde `main` para todo el desarrollo del pipeline. Esto permite:

- No alterar la rama principal durante el desarrollo.
- Activar el pipeline automáticamente ante cada push a la rama.
- Facilitar la revisión mediante Pull Requests antes de integrar a `main`.

```
# Crear la rama de trabajo
git checkout -b ci-cd-pipeline

# Trabajar en la rama
git add .
git commit -m "feat: implementar pipeline CI/CD"
git push origin ci-cd-pipeline
```

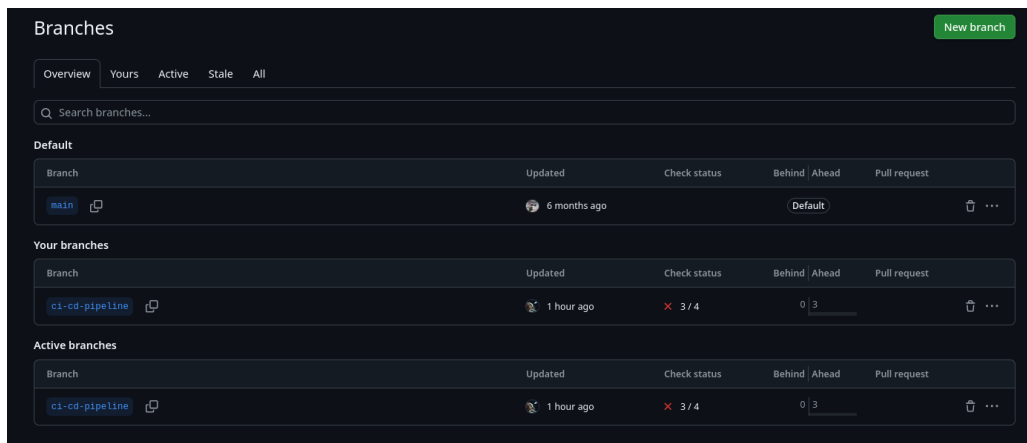


Figura 2: Estrategia de branching utilizada

6. Explicación del Pipeline CI/CD

El pipeline se compone de **4 jobs secuenciales** definidos en el archivo `.github/workflows/ci-cd.yml`.

1. **Build & Test** — Compilar y ejecutar pruebas
2. **Docker Build & Push** — Construir y subir imágenes Docker
3. **Deploy** — Desplegar en servidor remoto via SSH
4. **Notify** — Notificar resultado por email

6.1. Job 1: Build & Test

Este job se encarga de compilar tanto el backend como el frontend y ejecutar las pruebas unitarias del backend.

Pasos principales:

1. Clonar el repositorio usando `actions/checkout@v4`
2. Configurar JDK 17 con `actions/setup-java@v4` y cache de Maven
3. Compilar el backend: `./mvnw -B clean compile`
4. Ejecutar pruebas unitarias: `./mvnw -B test`
5. Empaquetar JAR: `./mvnw -B package -DskipTests`
6. Subir artefacto JAR con `actions/upload-artifact@v4`
7. Configurar Node.js 20 e instalar dependencias del frontend
8. Ejecutar lint y build del frontend con Vite

```

jobs:
  # =====
  # JOB 1: Build & Test
  # =====
  build-and-test:
    name: "🔧 Build & Test"
    runs-on: ubuntu-latest

    steps:
      # ---- Clonar repositorio ----
      - name: "📦 Clonar repositorio"
        uses: actions/checkout@v4

      # ---- Configurar Java 17 ----
      - name: "☕ Configurar JDK 17"
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: 'maven'

      # ---- Compilar Backend ----
      - name: "🔧 Compilar Backend (Spring Boot)"
        working-directory: ./backend
        run: ./mvnw -B clean compile -DskipTests

      # ---- Ejecutar Pruebas Unitarias ----
      - name: "🔧 Ejecutar Pruebas Unitarias"
        working-directory: ./backend
        run: ./mvnw -B test

      # ---- Empaquetar JAR ----
      - name: "📦 Empaquetar Backend (JAR)"
        working-directory: ./backend
        run: ./mvnw -B package -DskipTests

      # ---- Subir artefacto JAR ----
      - name: "📦 Subir artefacto JAR"
        uses: actions/upload-artifact@v4
        with:
          name: backend-jar
          path: backend/target/*.jar
          retention-days: 7

      # ---- Configurar Node.js 20 ----
      - name: "🟢 Configurar Node.js 20"
        uses: actions/setup-node@v4
        with:
          node-version: '20'

```

Figura 3: Configuración del Job Build & Test en el workflow YAML

6.2. Job 2: Docker Build & Push

Una vez que el Job 1 finaliza exitosamente, este job construye las imágenes Docker del backend y frontend, y las sube a Docker Hub.

Características:

- Login a Docker Hub usando secretos (DOCKERHUB_TOKEN)

- Docker Buildx para builds optimizados con cache
- Multi-stage builds: compilación separada del runtime
- Doble etiquetado: `latest` y SHA del commit

```
# Etiquetas generadas para cada imagen
lyoks/e-converse-backend:latest
lyoks/e-converse-backend:<sha-del-commit>
lyoks/e-converse-frontend:latest
lyoks/e-converse-frontend:<sha-del-commit>
```



```

# =====
# JOB 2: Docker Build & Push
# =====
docker-build-push:
  name: "🐳 Docker Build & Push"
  runs-on: ubuntu-latest
  needs: build-and-test

  steps:
    - name: "📁 Clonar repositorio"
      uses: actions/checkout@v4

    # ---- Login a Docker Hub ----
    - name: "🔑 Login a Docker Hub"
      uses: docker/login-action@v3
      with:
        username: ${ env.DOCKERHUB_USERNAME }
        password: ${ secrets.DOCKERHUB_TOKEN }

    # ---- Configurar Docker Buildx ----
    - name: "⚙️ Configurar Docker Buildx"
      uses: docker/setup-buildx-action@v3

    # ---- Build & Push Backend Image ----
    - name: "🐳 Build & Push Backend Image"
      uses: docker/build-push-action@v5
      with:
        context: ./backend
        file: ./backend/Dockerfile
        push: true
        tags: |
          ${ env.BACKEND_IMAGE }:latest
          ${ env.BACKEND_IMAGE }:${ github.sha }
        cache-from: type=gha
        cache-to: type=gha,mode=max

    # ---- Build & Push Frontend Image ----
    - name: "🐳 Build & Push Frontend Image"
      uses: docker/build-push-action@v5
      with:
        context: ./frontend-react
        file: ./frontend-react/Dockerfile
        push: true
        tags: |
          ${ env.FRONTEND_IMAGE }:latest
          ${ env.FRONTEND_IMAGE }:${ github.sha }
        build-args: |
          VITE_API_URL=/api

```

Figura 4: Configuración del Job Docker Build & Push

6.3. Job 3: Deploy

Este job despliega la aplicación en un servidor remoto (máquina virtual) mediante SSH. Utiliza la acción appleboy/ssh-action@v1 para conectarse al servidor y ejecutar comandos Docker.

Proceso de despliegue:

1. Conectar al servidor vía SSH con credenciales almacenadas en secretos
2. Crear/actualizar el archivo `docker-compose.yml` en el servidor

3. Descargar las imágenes más recientes (`docker pull`)
4. Detener los contenedores anteriores (`docker compose down`)
5. Levantar los nuevos contenedores (`docker compose up -d`)
6. Verificar el estado de los contenedores

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    needs: docker-build-push

    steps:
      - name: "📦 Clonar repositorio"
        uses: actions/checkout@v4

      # ---- Desplegar vía SSH a la VM ----
      - name: "🚀 Desplegar en servidor remoto"
        uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.DEPLOY_HOST }
          username: ${ secrets.DEPLOY_USER }
          password: ${ secrets.DEPLOY_PASSWORD }
          script: |
            echo "🚀 Iniciando despliegue de E-Converse..."
            sudo su

            # Crear directorio de trabajo si no existe
            mkdir -p ~/e-converse
            cd ~/e-converse

            # Crear/actualizar docker-compose.yml para producción
            cat > docker-compose.yml << 'COMPOSE_EOF'
            services:
              backend:
                image: lyoks/e-converse-backend:latest
                restart: unless-stopped
                environment:
                  SPRING_DATA_MONGODB_URI: $SPRING_DATA_MONGODB_URI
                ports:
                  - "8080:8080"

              frontend:
                image: lyoks/e-converse-frontend:latest
                restart: unless-stopped
                ports:
                  - "8081:80"
                depends_on:
                  - backend
            COMPOSE_EOF

            # Exportar variable de MongoDB
            export SPRING_DATA_MONGODB_URI="${ secrets.SPRING_DATA_MONGODB_URI }"

            # Descargar imágenes más recientes
            echo "📦 Descargando imágenes más recientes..."
            docker pull lyoks/e-converse-backend:latest
```

Figura 5: Configuración del Job Deploy vía SSH

6.4. Job 4: Notificaciones

El último job envía una notificación por correo electrónico informando el resultado de cada etapa del pipeline. Se ejecuta **siempre** (con `if: always()`), independientemente de si los jobs anteriores fueron exitosos o fallaron.

Información incluida en el correo:

- Repositorio, rama y commit
- Estado de cada job (éxito/fallo)
- Mensaje del commit
- Enlace directo a la ejecución en GitHub Actions
- Nombres de las imágenes Docker generadas

```
21 jobs:
22   notify:
23     name: "📧 Notificar resultado"
24     runs-on: ubuntu-latest
25     needs: [build-and-test, docker-build-push, deploy]
26     if: always()
27
28   steps:
29     - name: "📧 Enviar notificación por Email"
30       uses: dawidd6/action-send-mail@v3
31       with:
32         server_address: smtp.gmail.com
33         server_port: 587
34         username: "${{ secrets.MAIL_USERNAME }}"
35         password: "${{ secrets.MAIL_PASSWORD }}"
36         subject: |
37           [${{ needs.build-and-test.result == 'success' && needs.docker-build-push.result == 'success' && needs.deploy.result == 'success' && 🟢 ÉXITO }}]
38         to: "${{ secrets.NOTIFICATION_EMAIL }}"
39         from: "CI/CD E-Converse <${{ secrets.MAIL_USERNAME }}>"
40         body: |
41           =====
42           Pipeline CI/CD - E-Converse
43           =====
44
45           📁 Repositorio: "${{ github.repository }}"
46           🌿 Rama: "${{ github.ref_name }}"
47           📄 Commit: "${{ github.sha }}"
48           👤 Autor: "${{ github.actor }}"
49           🕒 Fecha: "${{ github.event.head_commit.timestamp }}"
50
51           =====
52           Resultados por etapa
53           =====
54
55           🛠 Build & Test: "${{ needs.build-and-test.result }}"
56           🐳 Docker Push: "${{ needs.docker-build-push.result }}"
57           🚀 Deploy: "${{ needs.deploy.result }}"
58
59           =====
60           Detalles
61           =====
62
63           Mensaje del commit:
64           "${{ github.event.head_commit.message }}"
65
66           Ver ejecución completa:
67           "${{ github.server_url }}/${{ github.repository }}/actions/runs/${{ github.run_id }}"
```

Figura 6: Configuración del Job de notificación por email

7. Pruebas unitarias implementadas

Se crearon pruebas unitarias para validar la lógica de negocio del backend usando **JUnit 5** y **Mockito**. Las pruebas mockean los repositorios de MongoDB para ejecutarse sin necesidad de una base de datos real.

7.1. Pruebas del ProductService

- Listar todos los productos
- Crear producto con y sin categoría
- Buscar producto por ID existente e inexistente
- Eliminar producto
- Verificación de precio y stock

7.2. Pruebas del CategoriaService

- Listar todas las categorías
- Crear nueva categoría
- Buscar categoría por ID
- Eliminar categoría
- Actualizar categoría existente

7.3. Pruebas del UsuarioService

- Listar usuarios con roles
- Crear usuario con contraseña encriptada (BCrypt)
- Buscar usuario por ID
- Actualizar usuario sin re-encriptar contraseña
- Eliminar usuario

7.4. Pruebas del JwtTokenUtil

- Generar token JWT válido (estructura de 3 partes)
- Extraer username del token
- Validar token con usuario correcto e incorrecto
- Manejo de tokens malformados e inválidos
- Tokens diferentes para diferentes usuarios

7.5. Pruebas del modelo Producto

- Constructor vacío y constructor completo
- Getters y setters de todos los campos
- Manejo de precios con decimales
- Manejo de stock cero

```
backend > src > test > java > backend > application > service > ProductoServiceTest.java > {} backend.application.service
25  class ProductoServiceTest {
41      void setUp() {
58          producto2.setDescripcion("Zapatillas Running");
59          producto2.setPrecio(new BigDecimal("349990"));
60          producto2.setStock(30);
61          producto2.setCategoria(categoria);
62      }
63
64      @Test
65      @DisplayName("Debe listar todos los productos")
66      void getProductos_debeRetornarListaDeProductos() {
67          when(productoRepository.findAll()).thenReturn(Arrays.asList(producto1, producto2));
68
69          List<Producto> resultado = productoService.getProductos();
70
71          assertNotNull(resultado);
72          assertEquals(2, resultado.size());
73          assertEquals("Nike Air Max", resultado.get(0).getNombre());
74          assertEquals("Adidas Ultraboost", resultado.get(1).getNombre());
75          verify(productoRepository, times(1)).findAll();
76      }
77
78      @Test
79      @DisplayName("Debe retornar lista vacia cuando no hay productos")
80      void getProductos_debeRetornarListaVacia() {
81          when(productoRepository.findAll()).thenReturn(List.of());
82
83          List<Producto> resultado = productoService.getProductos();
84
85          assertNotNull(resultado);
86          assertTrue(resultado.isEmpty());
87      }
88
89      @Test
90      @DisplayName("Debe crear un nuevo producto con categoria completa")
91      void nuevoProducto_debeCrearProductoConCategoria() {
92          when(categoriaService.buscarCategoria("cat1")).thenReturn(categoria);
93          when(productoRepository.save(any(Producto.class))).thenReturn(producto1);
94
95          Producto resultado = productoService.nuevoProducto(producto1);
96
97          assertNotNull(resultado);
98          assertEquals("Nike Air Max", resultado.getNombre());
99          assertEquals("Deportivos", resultado.getCategoria().getNombre());
100          verify(categoriaService, times(1)).buscarCategoria("cat1");
101          verify(productoRepository, times(1)).save(any(Producto.class));
102      }
}
```

Figura 7: Ejemplo de código de pruebas unitarias (ProductoServiceTest)

8. Gestión de secretos

GitHub Actions provee un mecanismo seguro para almacenar credenciales sensibles mediante **Repository Secrets**. Los secretos se configuran en **Settings** → **Secrets and variables** → **Actions** del repositorio.

8.1. Secretos configurados

Secreto	Descripción
DOCKERHUB_TOKEN	Token de acceso a Docker Hub para push de imágenes
SPRING_DATA_MONGODB_URI	URI de conexión a MongoDB Atlas
DEPLOY_HOST	IP del servidor de despliegue
DEPLOY_USER	Usuario SSH del servidor
DEPLOY_PASSWORD	Contraseña SSH del servidor
MAIL_USERNAME	Email remitente (Gmail) para notificaciones
MAIL_PASSWORD	Contraseña de aplicación de Gmail
NOTIFICATION_EMAIL	Email destinatario de las notificaciones

Cuadro 1: Secretos configurados en GitHub Actions

Características de seguridad:

- Los secretos se cifran antes de almacenarse en GitHub
- No se imprimen en los logs de ejecución (se enmascaran con ***)
- Solo están disponibles para los workflows del repositorio
- Se referencian con la sintaxis `${{ secrets.NOMBRE }}`

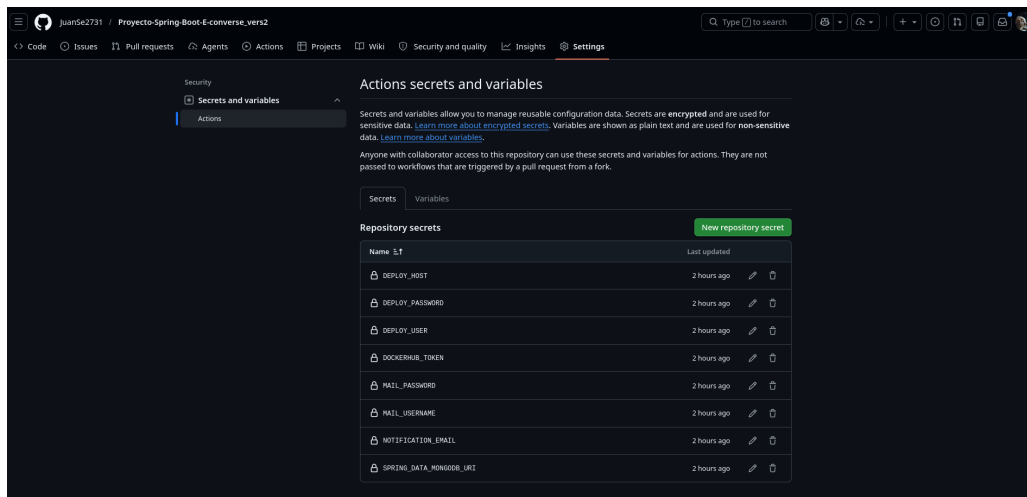


Figura 8: Secretos configurados en el repositorio de GitHub

9. Registro de contenedores (Docker Hub)

Las imágenes Docker se publican en **Docker Hub** bajo el usuario **lyoks**. Se utilizan dos repositorios de imágenes:

- lyoks/e-converse-backend — Imagen del backend (Spring Boot + JRE 17)
- lyoks/e-converse-frontend — Imagen del frontend (Nginx + archivos estáticos)

9.1. Estrategia de etiquetado

Cada build genera dos tags:

- latest — Siempre apunta a la versión más reciente
- <sha> — Tag inmutable con el SHA del commit, permite trazabilidad

9.2. Dockerfiles (Multi-stage builds)

Backend: Se utiliza un build en dos etapas: la primera compila el proyecto con Maven y la segunda ejecuta el JAR en un JRE mínimo.

```
backend > Dockerfile
1  # ---- Build stage ----
2  FROM maven:3.9.9-eclipse-temurin-17 AS builder
3  WORKDIR /app
4  COPY pom.xml .
5  RUN mvn -B -q dependency:go-offline
6  COPY src ./src
7  RUN mvn -B -q clean package -DskipTests
8
9  # ---- Runtime stage ----
10 FROM eclipse-temurin:17-jre
11 WORKDIR /app
12 COPY --from=builder /app/target/*.jar app.jar
13 EXPOSE 8080
14 ENTRYPOINT ["java", "-jar", "/app/app.jar"]
15
```

Figura 9: Dockerfile del Backend (multi-stage build)

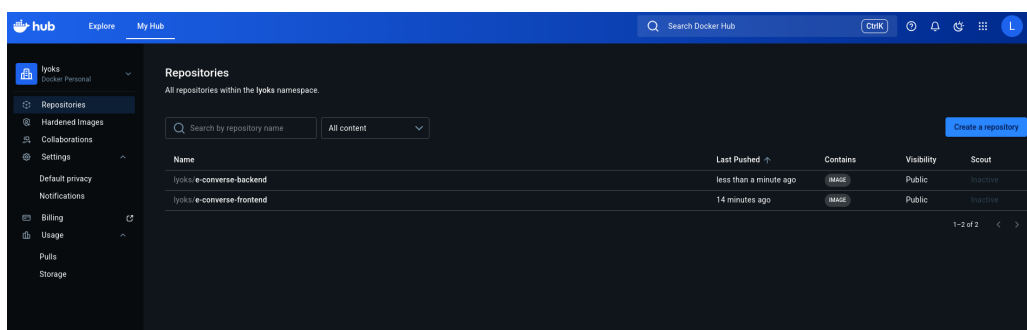
Frontend: Se compila el proyecto con Node.js/Vite y se sirve con Nginx Alpine.

```

frontend-react > Dockerfile
1  # ---- Build stage ----
2  FROM node:20 AS builder
3  WORKDIR /app
4  COPY package*.json ./
5  RUN npm install
6  COPY . .
7  ARG VITE_API_URL=/api
8  ENV VITE_API_URL=${VITE_API_URL}
9  RUN npm run build
10
11 # ---- Runtime stage ----
12 FROM nginx:1.27-alpine
13 COPY nginx.conf /etc/nginx/conf.d/default.conf
14 COPY --from=builder /app/dist /usr/share/nginx/html
15 EXPOSE 80
16 CMD ["nginx", "-g", "daemon off;"]

```

Figura 10: Dockerfile del Frontend (multi-stage build)



The screenshot shows the Docker Hub interface for a user named 'lyksa'. The left sidebar contains navigation links: Repositories, Hardened Images, Collaborations, Settings, Default privacy, Notifications, Billing, Usage, Pulls, and Storage. The main area is titled 'Repositories' and shows a list of repositories within the 'lyksa' namespace. There are two repositories listed: 'lyksa/e-converse-backend' and 'lyksa/e-converse-frontend'. The first repository was pushed 'less than a minute ago' and the second '14 minutes ago'. Both are public and have an 'IMAGE' icon. The interface also includes a search bar, a 'Create a repository' button, and a pagination indicator '1-2 of 2'.

Name	Last Pushed	Contains	Visibility	Scout
lyksa/e-converse-backend	less than a minute ago	IMAGE	Public	Inactive
lyksa/e-converse-frontend	14 minutes ago	IMAGE	Public	Inactive

Figura 11: Repositorios de imágenes en Docker Hub

10. Resultados obtenidos

10.1. Ejecución del pipeline completo

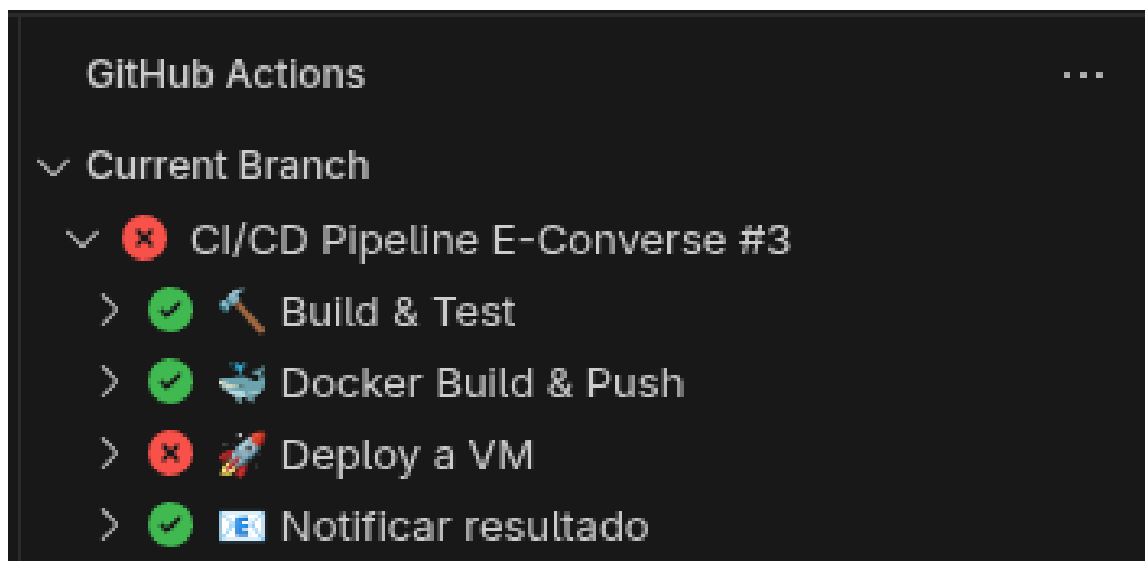


Figura 12: Pipeline completo ejecutado en GitHub Actions

10.2. Detalle de cada job

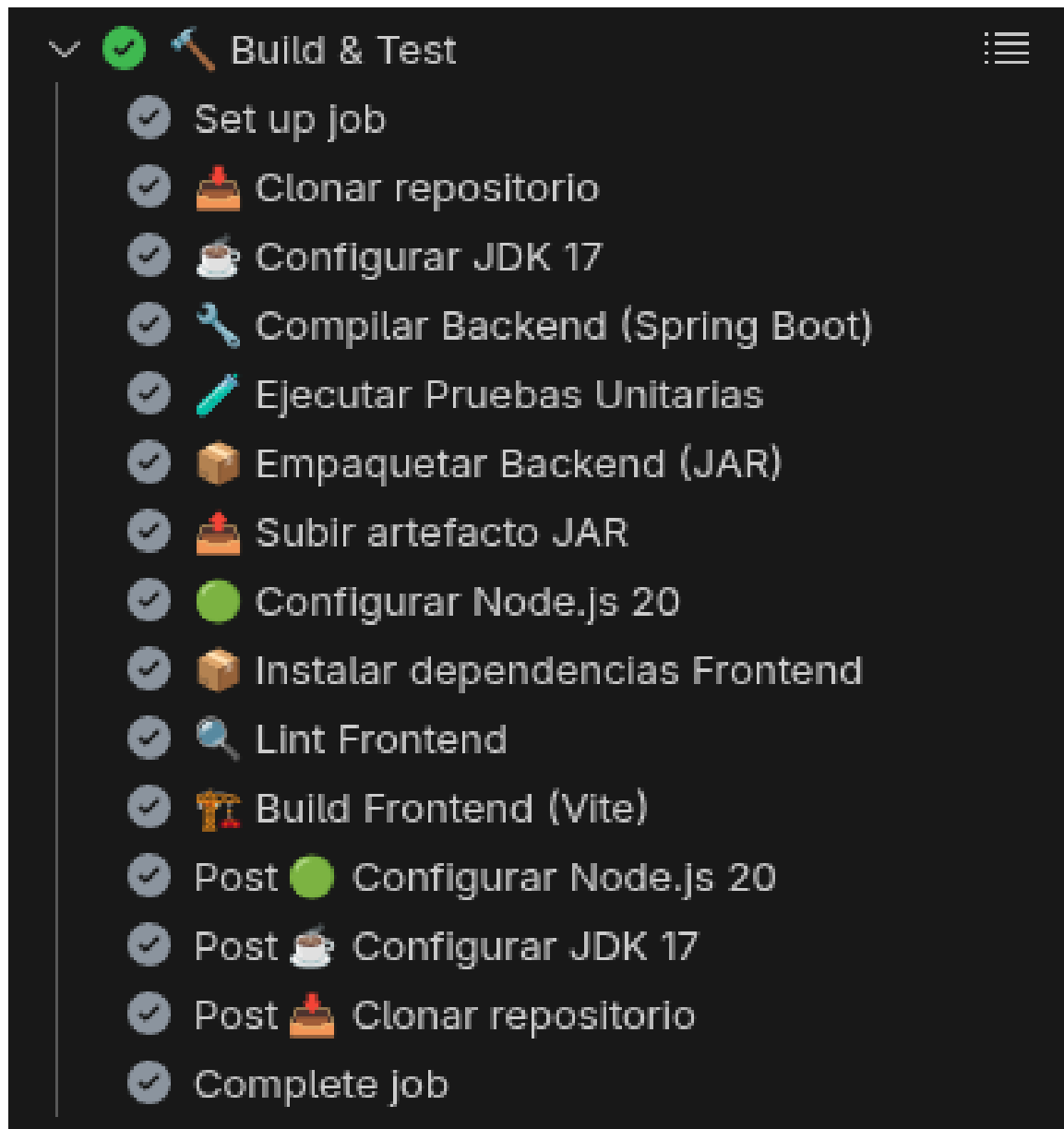


Figura 13: Detalle del Job Build & Test

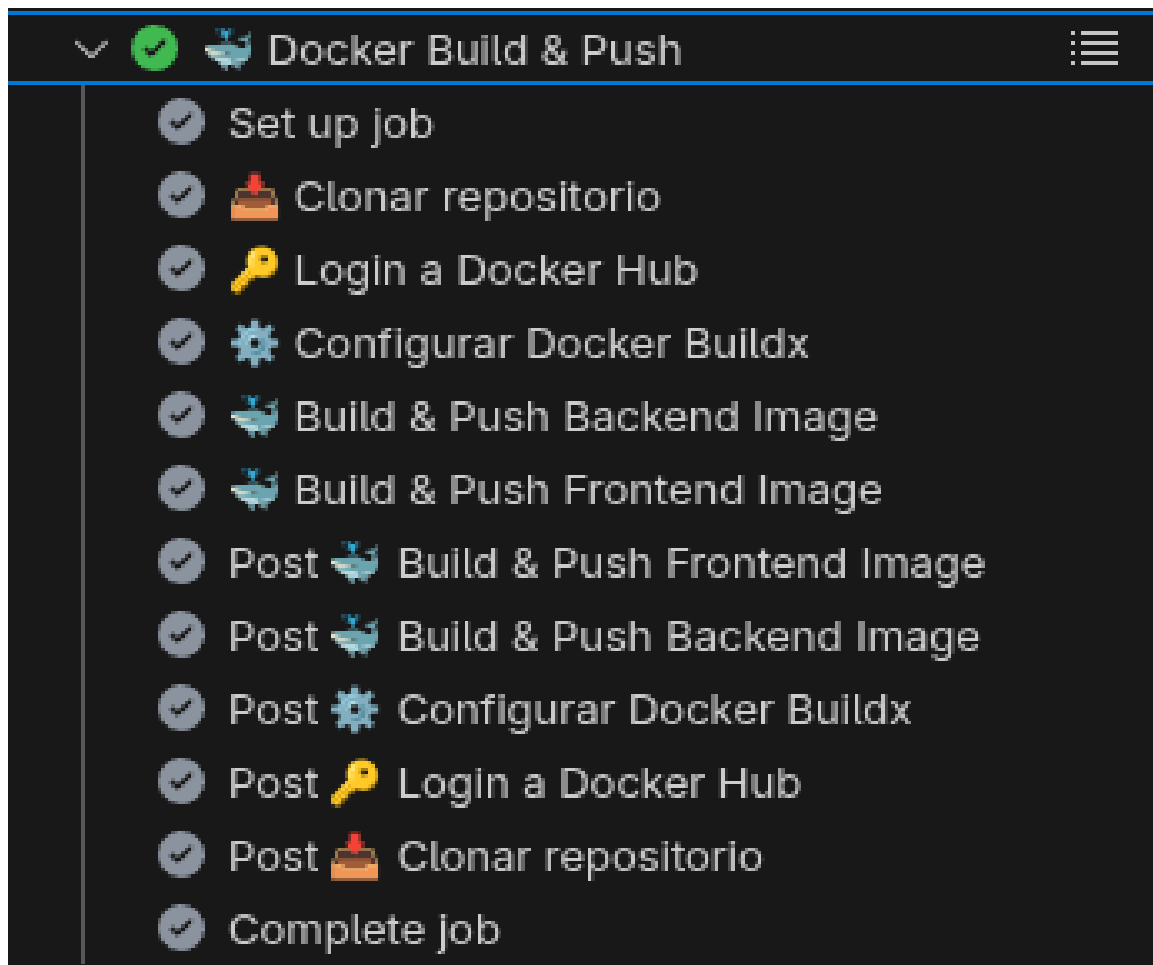


Figura 14: Detalle del Job Docker Build & Push

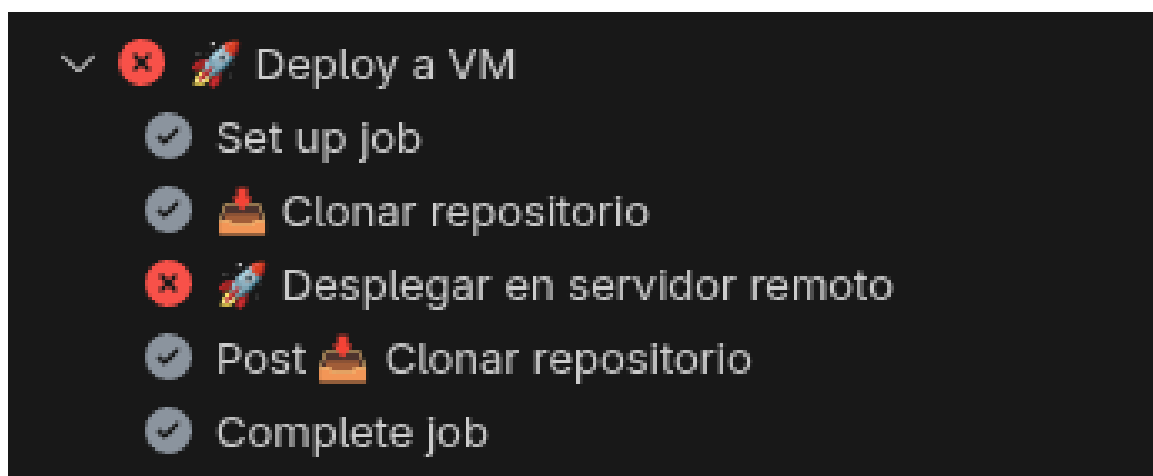


Figura 15: Detalle del Job Deploy

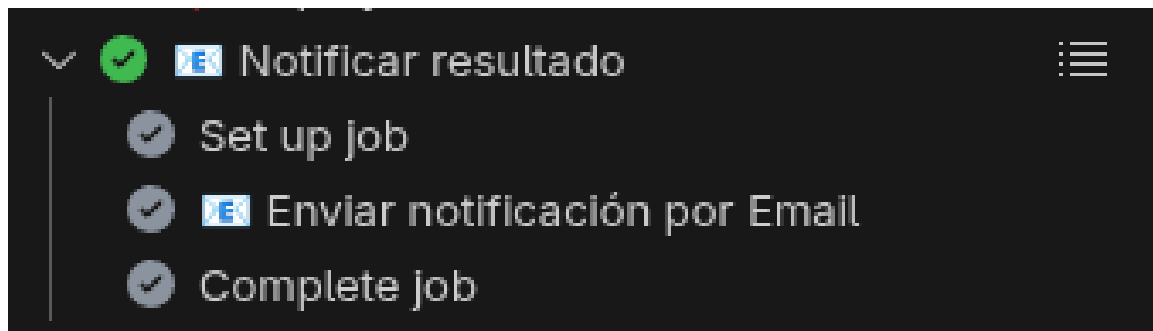


Figura 16: Detalle del Job Notify

10.3. Notificación recibida



Figura 17: Correo de notificación recibido

11. Análisis y conclusiones

11.1. Beneficios del CI/CD implementado

- **Automatización completa:** Desde el commit hasta el despliegue, sin intervención manual.
- **Detección temprana de errores:** Las pruebas unitarias se ejecutan en cada push, evitando que código defectuoso llegue a producción.
- **Trazabilidad:** El etiquetado de imágenes con SHA permite rastrear exactamente qué commit está desplegado.
- **Seguridad:** Los secretos se gestionan de forma cifrada mediante GitHub Secrets.
- **Visibilidad:** Las notificaciones por email mantienen al equipo informado del estado del pipeline.

- **Reproducibilidad:** El uso de Docker garantiza que la aplicación se ejecute de forma idéntica en cualquier entorno.

11.2. Lecciones aprendidas

- La configuración de secretos en GitHub Actions es fundamental para proteger credenciales sensibles.
- Los multi-stage builds de Docker reducen significativamente el tamaño de las imágenes finales.
- La estrategia de branching (GitHub Flow) facilita la integración del pipeline con el flujo de desarrollo.
- Las notificaciones por email permiten reaccionar rápidamente ante fallos en el pipeline.
- Hay que tener en cuenta que Github Actions no es un servicio self-hosted, por lo que no se puede ejecutar el despliegue en un servidor fuera de redes públicas.